

PART 1 - Problem 1

Tag List: [STANDARD ASSESSMENT] [PARTIAL CREDIT] [SYMBOLIC MATH TOOLBOX] [CONTROL TOOLBOX]

Consider the linear system described by the state equations:

$$\begin{cases} \dot{x}_1(t) &= 2 \beta x_1(t) + 3 x_2(t) + u(t) \\ \dot{x}_2(t) &= -3 x_1(t) + 2 \beta x_2(t) \\ \dot{x}_3(t) &= -2 x_3(t) \\ y(t) &= x_2(t) \end{cases}$$

where $\beta \in \mathbb{R}$ is a generic scalar and

$$\begin{bmatrix} x_1(0) \\ x_2(0) \\ x_3(0) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Assignment

Your code shall

- Declare the complex variable s and the symboli real parameter β as the **symbolic variables** **s** and **p_beta**;
- Determine the **characteristic polynomial** $p_A(s) = \det(sI - A)$ as a symbolic function of the complex variable s and the real parameter β : $p_A(s, \beta) = \det(sI - A)$ and assign the expression to the **symbolic function** **pA(s, p_beta)**.
- Generate a set of 100 different values, logarithmically spaced, for the parameter β in the interval $\left[-1000, -\frac{1}{1000}\right]$ and assign these values to the numeric **row vector** **beta_n**; generate a set of 100 different values, logarithmically spaced, for the parameter β in the interval $\left[+\frac{1}{1000}, +1000\right]$ and assign these values to the numeric **row vector** **beta_p**.
- Substitute each value of β , starting with the smallest and ending with the largest, in the symbolic function **pA(s, p_beta)**, then compute the eigenvalues of the matrix as roots of the resulting polynomial $p(s)$ using the function `solve()`. For each value of β , assign the result to the column vector **eigAbeta** and sort them using the command `sort(eigAbeta, 'ascend')`. Insert the resulting sorted column vector in the **symbolic** array **eigA_b**, as a column element. At the end, **eigA_b** must be a matrix with 200 columns and 3 rows.
- Letting $\beta = -1$ and $u(t) = 3 \forall t$, calculate the corresponding **equilibrium state** and **equilibrium output**. Store the results respectively using **xEQ** as the symbolic column vector of the equilibrium state and **yEQ** as the symbolic output value.

Solution

```

% let's declare the needed symbolic variables
% -->> 1st bullet point of the assignment
syms s p_beta

% =====
% IMPORTANT: you can not use the name "beta" as variable name, because
% "beta" is a reserved name, corresponding to the Matlab function Y = beta(Z,W)
% Such a function computes the beta function for corresponding elements of
% Z and W.
% Re-using the reserved name "beta" for defining a whatever variable is not
% allowed in the Matlab Grader environment.
% =====

% --- the symbolic LTI system matrices ---
A = [2*p_beta, 3, 0;
     -3, 2*p_beta, 0;
     0, 0, -2];

% Note:
%     We don't need to declare matrix A as a symbolic matrix
%     because it has been built up using symbolic variables,
%     so it automatically becomes a symbolic matrix variable.
%
%     However, we need to declare matrices B and C
%     as symbolic; otherwise, they will be numeric.
%

B = sym([1; 0; 0]);
C = sym([0, 1, 0]);
% --- the symbolic LTI system matrices ---

% --- the characteristic polynomial of the matrix A ---
%     in terms of powers of the symb. variable "s"
%
%     Remark-1: p_beta is a parameter,
%               whereas s is the polynomial variable.
%
%     Remark-2: Declaring pA as pA(s, p_beta)
%               automatically makes pA a symbolic function
%               of s and p_beta.
%
% --->> 2nd bullet point of the assignment
pA(s, p_beta) = collect(det(s*eye(size(A))-A),s)

```

$$pA(s, p_beta) = s^3 + (2 - 4p_\beta)s^2 + (4p_\beta^2 - 8p_\beta + 9)s + 8p_\beta^2 + 18$$

```

% -- creation of the numeric row-vectors beta_n and beta_p --
% ---->> 3rd bullet point of the assignment
%
% excerpt from "help logspace" :
% logspace Logarithmically spaced vector.
% logspace(X1, X2) generates a row vector of 50 logarithmically

```

```

%   equally spaced points between decades 10^X1 and 10^X2.
%
%   logspace(X1, X2, N) generates N points.
%   For N = 1, logspace returns 10^X2.
%
% Thus: 1/1000 <--> 10^(-3) ; 1000 <--> 10^(+3)
%
% --- beta_p: logspace(X1, X2, N) <--> logspace(-3, +3, 100)
beta_p = logspace(-3,3,100);
% --- beta_p

% --- beta_n: given X1 and X2 as real values, then the output of the
%             function logspace() is a vector fo real positive values; in
%             order to obtain the requested negative values for the
%             vector beta_n, we should change the sign of the logspace()
%             output. Moreover, we have to sort the resulting negative
%             values in ascending order.
beta_n = sort(-logspace(-3,3,100));
% ---->> 3rd bullet point of the assignment
% -- creation of the numeric row-vectors beta_n and beta_p --

% Now we can merge the row vectors, obtaining an unique row vector, sorted
% in ascending order.
beta_val = [beta_n beta_p];
% Remark: this is not a mandatory step, but it can be helpful when
%         answering to the next questions.

% Preallocate the matrix eigA_b, filling up it with zeros.
% Remark: this is not a requested assignment step,
%         but preallocation allows for higher execution speed.
eigA_b = sym(zeros(3, numel(beta_val)));

% ---- Evaluation of the eigenvalues corresponding
%         to the different values of p_beta ----
% ----->> 4th bullet point
for n=1:numel(beta_val)
    currBETA = beta_val(n); % let's extract a value for p_beta
                        % starting from the minimum value
                        % up to the max value
    curr_pA = pA(s, currBETA); % let's substitute in pA(s, p_beta); now
                        % curr_pA is a symbolic expression of the
                        % complex variable s
    eigAbeta = solve(curr_pA); % solving the expression,
                        % looking for the polynomial roots, i.e. the
                        % eigenvalues corresponding tto the
                        % parameter value currBETA
    eigAbeta = sort(eigAbeta, 'ascend'); % sort the solution in ascending order:
                        % for each value of currBETA we
                        % have 3 eigenvalues.
    eigA_b(:,n) = eigAbeta; % store the sorted eigenvalues in the final data-structure
end % for n
% ----->> 4th bullet point

```

```
% ---- Evaluation of the eigenvalues corresponding
%       to the different values of p_beta ----
```

```
% ----->> 5th bullet point
% --- Equilibrium state and equilibrium output ---
A1 = subs(A,p_beta, -1) % as requested,
```

```
A1 =
```

$$\begin{pmatrix} -2 & 3 & 0 \\ -3 & -2 & 0 \\ 0 & 0 & -2 \end{pmatrix}$$

```
% assign the value "-1"
% to the parameter p_beta and evaluate the
% corresponding matrix A
```

```
barU = 3; % define the constant equilibrium input signal
% solve A1*xEQ + B*barU = 0 <--> A1*xEQ = -B*barU = b
b = -B*barU;
% compute the equilibrium state xEQ as the solution of the matrix
% equation A1*xEQ = b ----> using linsolve
xEQ = linsolve(A1,b)
```

```
xEQ =
```

$$\begin{pmatrix} \frac{6}{13} \\ -\frac{9}{13} \\ 0 \end{pmatrix}$$

```
% finally, compute the corresponding equilibrium output
yEQ = C*xEQ
```

```
yEQ =
```

$$-\frac{9}{13}$$

```
% --- Equilibrium state and equilibrium output ---
% ----->> 5th bullet point
```